

9+b

Returns: If the assertion-control object of the violated contract assertion satisfies `queryable_label([support.contract.control.query])`, the result of invoking `query(key, index)` on the assertion-control object; otherwise, `nullptr`.

17.10.(4 + a) Assertion-control concepts and utilities [support.contract.control]

Add a new section 17.10.(4+a) ([support.contract.control]) after [support.contract.invoke]

17.10.(4+a) Assertion-control concepts and utilities[support.contract.control]

This section defines a mix of concepts and language definitions for library-only assertion control types. Where the concept represents a core-language assertion control type we reference back to [basic.contract.control]

17.10.(4 + a).1 General [support.contract.control.general]

17.10.(4+a).1 General [support.contract.control.general]

1 This subclause defines concepts, types, and utilities related to controlling the behavior of contract assertions([basic.contract.control]). All concepts and variable templates defined in this subclause are declared in the namespace `std::contracts::labels`.

2 For each concept *C* defined in this subclause, a corresponding variable template is provided:

```
namespace std::contracts::labels {
    template<auto control_object>
        constexpr bool is_C = C<decltype(control_object)>;
}
```

17.10.(4 + a).2 Concept `assertion_control_object` [support.contract.control.aco]

17.10.(4+a).2 Concept `assertion_control_object`[support.contract.control.aco]

```
template<class T>
    concept assertion_control_object =
        requires { typename T::assertion_control_object; };
```

1 The `assertion_control_object` concept specifies the base requirement for all assertion-control objects.

[*Note:* Every assertion-control object must satisfy this concept. As a general convention, label types name themselves with this member typedef. — *end note*]

17.10.(4 + a).3 Concept `allowed_semantics_label` [support.contract.control.allowed]

17.10.(4+a).3 Concept `allowed_semantics_label` [support.contract.control.allowed]

```
template<class T>
    concept allowed_semantics_label =
        assertion_control_object<T> &&
        requires (const T t) {
            requires is_const_v<decltype(T::allowed_semantics)>;
            evaluation_semantic_set({t.allowed_semantics});
        }
```

};

1 A type `T` models `allowed_semantics_label` if it is an allowed-semantics control type(`[basic.contract.control]`).

2 [Note: Combined assertion-control objects intersect the allowed semantics sets of their constituents. — *end note*]

17.10.(4 + a).4 Concept `semantic_computation_label` [`support.contract.control.compute`]

17.10.(4+a).4 Concept `semantic_computation_label` [`support.contract.control.compute`]

```
template<class T>
concept semantic_computation_label =
    assertion_control_object<T> &&
    requires (const T t, evaluation_semantic s) {
        { t.compute_semantic(s) } -> same_as<evaluation_semantic>;
    };
```

1 A type `T` models `semantic_computation_label` if it is a computed-semantics control type(`[basic.contract.control]`).

2 [Note: Combined assertion-control objects chain their `compute_semantic` functions from left to right. — *end note*]

17.10.(4 + a).5 Concept `dimensioned_label` [`support.contract.control.dim`]

17.10.(4+a).5 Concept `dimensioned_label` [`support.contract.control.dim`]

```
template<class T>
concept dimensioned_label =
    assertion_control_object<T> &&
    is-specialization-of<typename T::dimensions, dimension_list>;
```

1 A type `T` models `dimensioned_label` if it declares dimensions via a nested `dimensions` member that is a specialization of `dimension_list`.

2 A type is a *dimensioned control type* if it models `dimensioned_label`.

3 [Note: Combined assertion-control objects have the union of the dimensions of their constituent objects, and it is ill-formed if the dimensions of their constituents have a non-empty intersection. — *end note*]

17.10.(4 + a).6 Concept `local_violation_label` [`support.contract.control.lv1`]

17.10.(4+a).6 Concept `local_violation_label` [`support.contract.control.lv1`]

```
template<class T>
concept local_violation_label =
    assertion_control_object<T> &&
    requires (remove_const_t<T> t, const contract_violation& v) {
        t.handle_contract_violation(v);
        requires is_same_v<decltype(t.handle_contract_violation(v)), void>
            || is_convertible_v<decltype(t.handle_contract_violation(v)),
                               violation_handled>;
    };
```

1 A type `T` models `local_violation_label` if it has a member function `handle_contract_violation` that accepts a `const contract_violation&`. The return type shall be `void` or a type convertible to `violation_handled`.

2 When a contract violation occurs for an assertion whose control object satisfies this concept, `handle_contract_violation` is invoked before the global contract-violation handler (`[basic.contract.handler]`). If the return type is not `void` and the return value converts to `violation_handled::handled`, the global handler is not invoked.

[*Note*: Combined assertion-control objects chain their `handle_contract_violation` members from right to left. — *end note*]

17.10.(4 + a).7 **Concept `identification_label`** [support.contract.control.ident]

17.10.(4+a).7 Concept `identification_label` [support.contract.control.ident]

```
template<class T>
concept identification_label =
    assertion_control_object<T> &&
    requires (const T t) {
        { t.group_names } -> ranges::range;
        { *(ranges::begin(t.group_names)) }
            -> convertible_to<string_view>;
    };

```

1 A type `T` models `identification_label` if it has a `group_names` member that is a range of values convertible to `string_view`.

2 A type is a *identification control type* if it models `identification_label`.

3 [*Note*: The value of an identification label's `group_names` can be used as input to the implementation-defined choice of evaluation semantic when evaluating a contract assertion. (`[basic.contract.eval]`). — *end note*]

4 [*Note*: Combined assertion-control objects produce the sorted, unique union of the group name sets of their constituents. — *end note*]

17.10.(4 + a).8 **Concepts `compute_comment_label`** [support.contract.control.comment]

17.10.(4+a).8 Concepts `compute_comment_label` [support.contract.control.comment]

```
template<class T>
concept compute_comment_label =
    assertion_control_object<T> &&
    requires (T t, const char* comment) {
        { t.compute_comment(comment) } -> convertible_to<const char*>;
    };

```

1 Types satisfying `compute_comment_label` have a `constexpr` member function `compute_comment` that are used to transform at compile time the `comment` fields of the `contract_violation` object (`[support.contract.violation]`) before they are stored.

2 A type is a *computed-comment control type* if it models `computed_comment_label`.

[*Note*: Combined assertion-control objects chain their `compute_comment` functions from left to right. — *end note*]

17.10.(4 + a).9 **Concept `queryable_label`** [support.contract.control.query]

17.10.(4+a).9 Concept `queryable_label` [support.contract.control.query]

```
template<class T>
concept queryable_label =
    assertion_control_object<T> &&
    requires (const T t, const void* key, size_t index) {
        { t.query(key, index) } -> same_as<void*>;
    };
```

1 A type `T` models `queryable_label` if it has a `query` member function providing a type-erased interface for the contract-violation handler to query data from the assertion-control object at runtime. The `key` parameter identifies the data to query; the `index` parameter allows returning multiple results for the same key. `query` returns `nullptr` if no result is available for the given key and index.

2 A type `T` is a *queryable control type* if it models `queryable_label`.

3 [*Note*: Combined assertion-control objects delegate `query` to their constituents, using index arithmetic to iterate through results from both. — *end note*]

17.10.(4 + a).10 **Enum `violation_handled`** [support.contract.control.vhandled]

17.10.(4+a).10 Enum `violation_handled` [support.contract.control.vhandled]

```
namespace std::contracts {
    enum class violation_handled {
        not_handled,
        handled
    };
}
```

1 The type `violation_handled` is used as a return type by local violation handlers([support.contract.control.lv]) to indicate whether a contract violation has been fully handled.

17.10.(4 + a).11 **Class `evaluation_semantic_set`** [support.contract.control.semset]

17.10.(4+a).11 Class `evaluation_semantic_set` [support.contract.control.semset]

17.10.(4 + a).11.1 **General** [support.contract.control.semset.general]

17.10.(4+a).11.1 General [support.contract.control.semset.general]

```
namespace std::contracts {
    class evaluation_semantic_set {
    public:
        // [support.contract.control.semset.cons], constructors
        constexpr evaluation_semantic_set();
        constexpr evaluation_semantic_set(const evaluation_semantic_set&);
        constexpr evaluation_semantic_set(initializer_list<evaluation_semantic>);
```